

Extender el Monolito Mínimo con Operaciones en Singular

Guía de laboratorio

1. Descripción general

En esta práctica se trabajará sobre el *monolito mínimo* desarrollado con Node.js y Express, extendiéndolo para soportar operaciones en singular sobre recursos de tipo **producto** y **pedido**. El objetivo es que se implementen rutas REST del tipo GET/PUT/DELETE para un solo recurso identificado por su **id**, y que investigue cómo manejar parámetros de ruta y códigos de estado HTTP en Express.¹

2. Punto de partida

Se asume que el proyecto base ya está creado siguiendo la guía de *Monolito mínimo con Node.js y Express*. En particular, se cuenta con:

- Un archivo `index.js` que:
 - Configura un servidor Express y aplica el middleware `express.json()` para leer cuerpos JSON.
 - Define un arreglo `productos` con al menos tres productos de ejemplo.
 - Define un arreglo `pedidos` vacío para ir almacenando pedidos en memoria.
 - Implementa las rutas:
 - GET `/productos` para listar todos los productos.
 - POST `/pedidos` para crear un nuevo pedido, con una validación básica del cuerpo de la petición.
 - GET `/pedidos` para listar todos los pedidos registrados.
 - Arranca el servidor con `app.listen(PORT, ...)`.

3. Parte A: Investigación

Previo a la implementación se deberá investigar los siguientes puntos:

1. ¿Cómo se definen rutas con parámetros en Express?
2. ¿Cómo se accede al valor de esos parámetros en el manejador de la ruta
3. ¿Qué códigos de estado HTTP se usan típicamente para los siguientes casos?

- Recurso encontrado correctamente.

¹El monolito mínimo original concentra datos en memoria, lógica de negocio básica, rutas HTTP y arranque del servidor en un solo archivo `index.js`.

- Recurso creado exitosamente.
- Recurso no encontrado.
- Petición inválida (datos incorrectos o incompletos).
- Recurso eliminado sin contenido en la respuesta.

4. Parte B: Extender el monolito mínimo

En el mismo archivo `index.js` (sin dividir en más módulos), extender la API para soportar operaciones en singular:

4.1. Ruta GET /producto/:id

Agregar una ruta que permita obtener un solo producto, identificado por su `id`:

- Leer el parámetro `id` y convertirlo a número (por default son cadenas en un petición HTTP).
- Buscar en el arreglo `productos` el elemento cuyo `id` coincida con el valor recibido.
- Si se encuentra, responder con el objeto producto y código de estado que corresponda
- Si no se encuentra, responder con un mensaje de error en JSON y código de estado que corresponda

4.2. Ruta GET /pedido/:id

Agregar una ruta que permita obtener un solo pedido, identificado por su `id`:

- Leer `id`
- Buscar el pedido correspondiente en el arreglo `pedidos`.
- Responder con el pedido si existe y el código correspondiente.
- Responder con un mensaje de error si no existe y el código correspondiente.

4.3. Ruta PUT /producto/:id

Agregar una ruta para actualizar un producto existente:

- Leer `id` y los nuevos datos
- Localizar el producto correspondiente en el arreglo.
- Si no existe, responder con el código y mensaje correspondiente.
- Si existe:
 - Aplicar una validación básica: por ejemplo, si se envía `precio`, verificar que sea un número positivo; si no lo es, responder con el código correspondiente
 - Actualizar únicamente los campos enviados (por ejemplo, `nombre` y/o `precio`), pueden no enviarse todos.
 - Responder con el producto actualizado y código que corresponda

4.4. Ruta DELETE /pedido/:id

Agregar una ruta para eliminar un pedido existente:

- Leer `id`
- Buscar el índice del pedido en el arreglo `pedidos` (por ejemplo, con `findIndex`).
- Si no existe, responder con mensaje y código correspondiente.
- Si existe:
 - Eliminar el elemento del arreglo.
 - Responder con código que corresponda (puede haber alternativas) y un mensaje de confirmación.

5. Parte C: Pruebas de la API

Utilizando `curl`, Postman, Insomnia u otro cliente HTTP, realizar al menos las siguientes pruebas:

1. GET /producto/:id

- Caso exitoso: consultar un `id` correspondiente a un producto existente.
- Caso de error: consultar un `id` inexistente y verificar la respuesta

2. GET /pedido/:id

- Crear primero uno o más pedidos con `POST /pedidos`.
- Consultar un `id` existente y uno inexistente.

3. PUT /producto/:id

- Actualizar un producto con datos válidos (por ejemplo, cambiar el `nombre` o el `precio`).
- Intentar actualizar un producto inexistente
- Enviar datos inválidos (por ejemplo, un `precio` negativo)

4. DELETE /pedido/:id

- Eliminar un pedido existente y verificar el código de estado
- Intentar eliminar nuevamente el mismo `id` y observar la respuesta que se obtiene.